

auto_ml 사용 설명서

이 문서는 `auto_ml` 라이브러리를 처음 사용하는 분을 위해 단계별로 친절하게 정리한 가이드입니다. 명령어 한 줄씩 따라 치면 학습 → 리포트 → 스코어링까지 끝까지 완주할 수 있도록 구성했습니다.

목차

1. 이 라이브러리는 무엇인가요?
 2. 전체 흐름 한눈에 보기
 3. 사전 준비 (Prerequisites)
 4. 설치하기
 5. 빠른 시작: 더미 데이터로 5분 만에 돌려보기
 6. 실제 데이터로 사용하기
 7. 설정 파일(YAML) 이해하기
 8. `features.csv` 작성법
 9. 산출물(결과물) 확인하기
 10. 타이타닉 데이터로 검증해 보기
 11. 자주 묻는 질문 / 문제 해결
 12. 폐쇄망(인터넷이 안 되는 환경) 이관
 13. 용어 사전
-

1. 이 라이브러리는 무엇인가요?

`auto_ml` 은 이진 분류(0 또는 1을 맞히는 문제) 를 자동으로 학습하고 점수를 매기는 사내용 라이브러리입니다.

예를 들어, - 어떤 고객이 이탈할지(1) / 안 할지(0) - 어떤 거래가 사기인지(1) / 정상인지(0) - 어떤 사용자가 상품을 살지(1) / 안 살지(0)

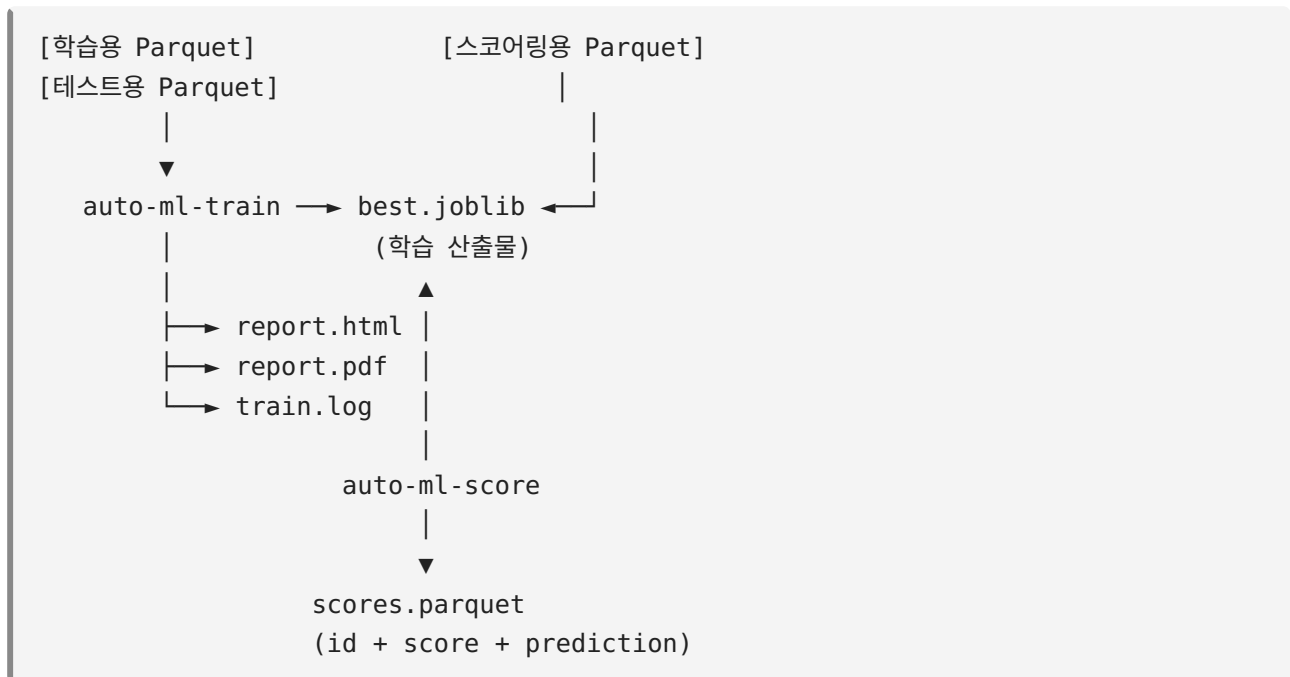
같은 "예/아니오" 문제를 푹니다.

복잡한 ML 코드를 직접 짜지 않아도 됩니다. **YAML 설정 파일 한 개와 컬럼 정의 CSV 한 개**만 작성하면, 라이브러리가 알아서:

- 결측치를 채우고, 이상치를 자르고, 스케일을 맞추고
- LightGBM / XGBoost / CatBoost 세 가지 모델을 동시에 학습하고
- Optuna 로 하이퍼파라미터를 자동 튜닝하고
- 가장 성능이 좋은 모델을 골라서
- HTML / PDF 리포트까지 만들어 줍니다.

학습이 끝난 뒤에는 **auto-ml-score** 명령 한 번으로 새 데이터에 점수를 매길 수 있습니다 (cron 등에 걸어 매일 돌리는 용도).

2. 전체 흐름 한눈에 보기



핵심은 단 두 개의 명령어입니다:

명령어	언제 쓰나요?	결과물
<code>auto-ml-train</code>	모델을 만들고 싶을 때 (보통 1회)	<code>best.joblib</code> , 리포트
<code>auto-ml-score</code>	새 데이터에 점수를 매기고 싶을 때 (반복)	<code>scores.parquet</code>

3. 사전 준비 (Prerequisites)

필요한 것

- **Python 3.9 이상** (`python --version` 으로 확인)
- **pip** (보통 Python 과 함께 설치됩니다)
- **Git** (소스 코드를 받을 때 필요)
- (선택) **WeasyPrint 시스템 의존성** — PDF 리포트를 만들 때 필요합니다.
- macOS: `brew install pango libffi`
- Ubuntu/Debian: `sudo apt-get install libpango-1.0-0 libpangoft2-1.0-0`
- PDF 가 안 만들어져도 학습/스코어링은 정상 동작합니다 (HTML 리포트는 그대로 생성).

디스크 공간 / 메모리

- 데이터 크기에 따라 다르지만, 더미 예제 기준 **500MB** 이상이면 충분합니다.
- 학습 데이터가 수백만 행이라면 **8GB 이상 메모리** 를 권장합니다.

4. 설치하기

터미널을 열고 프로젝트 폴더로 이동한 뒤, 아래 명령을 **한 줄씩** 차례대로 실행하세요.

```
# 1) 프로젝트 폴더로 이동
cd auto-ml-briefly

# 2) 가상환경 생성 (시스템 Python 을 더럽히지 않기 위해)
python -m venv .venv

# 3) 가상환경 활성화
#   macOS / Linux
source .venv/bin/activate
#   Windows (PowerShell)
.venv\Scripts\Activate.ps1

# 4) 의존성 패키지 설치
pip install -r requirements.txt

# 5) auto_ml 자체를 editable 모드로 설치
#   (auto-ml-train, auto-ml-score 명령이 활성화됩니다)
pip install -e .
```

설치 확인

```
auto-ml-train --help
auto-ml-score --help
```

도움말이 출력되면 성공입니다. `command not found` 가 뜨면 **가상환경이 활성화되어 있는지** 다시 확인해 주세요 (프롬프트 앞에 `(.venv)` 가 보여야 합니다).

5. 빠른 시작: 더미 데이터로 5분 만에 돌려보기

진짜 데이터를 준비하기 전에, **라이브러리가 잘 동작하는지** 먼저 확인합니다.

```
# 1) 더미 데이터 생성 (data/ 폴더에 train/test/score_input parquet 파일이 만들어집니다)
python examples/make_dummy_data.py

# 2) 학습 실행
auto-ml-train --config configs/example.yaml

# 3) 스코어링 실행
auto-ml-score --config configs/example.yaml
```

무슨 일이 일어났는지 확인

```
artifacts/
├── models/
│   └── best.joblib          ← 학습된 모델 + 전처리기 + 메타데이터 한 덩어리
├── reports/
│   ├── report.html        ← 브라우저로 열어보세요
│   └── report.pdf          ← 동일 내용 PDF
├── scores/
│   └── scores.parquet      ← user_id + score + prediction
└── logs/
    ├── train_YYYYMMDD_HHMMSS.log
    └── score_YYYYMMDD_HHMMSS.log
```

artifacts/reports/report.html 을 브라우저로 열어 보면 모델 비교표, ROC 곡선, feature importance 등이 보일 겁니다. **여기까지 됐다면 설치는 끝!**

6. 실제 데이터로 사용하기

이제 본인 데이터로 돌려봅시다. 필요한 것은 단 4개입니다.

6-1. 준비물 체크리스트

- **[] 학습 데이터 Parquet 파일** (예: data/my_train.parquet)
- 타겟 컬럼은 **0 또는 1** 만 있어야 합니다.
- **[] 테스트 데이터 Parquet 파일** (예: data/my_test.parquet)
- 학습 데이터와 같은 컬럼 구조여야 합니다.
- **[] 스코어링 입력 Parquet 파일** (예: data/my_score_input.parquet)
- 타겟 컬럼은 없어도 됩니다 (있어도 무시).
- **[] features.csv** — 어떤 컬럼을 쓸지 정의 (다음 섹션에서 설명).

6-2. 설정 파일 복사

```
cp configs/example.yaml configs/my_config.yaml
```

`configs/my_config.yaml` 을 열고 다음 부분을 본인 환경에 맞게 수정합니다:

```
train_data_path: ./data/my_train.parquet      # ← 변경
test_data_path:  ./data/my_test.parquet       # ← 변경
target_column:   target                       # ← 본인 타겟 컬럼명으로
features_csv:    ./configs/my_features.csv    # ← features.csv 경로
id_columns:
  - user_id                                     # ← 결과에 보존할 식별자
artifact_dir:    ./artifacts/models

scoring:
  input_path:     ./data/my_score_input.parquet # ← 변경
  output_path:    ./artifacts/scores/scores.parquet
  id_columns:     [user_id]
  threshold:      0.5
```

6-3. 학습 → 스코어링 실행

```
auto-ml-train --config configs/my_config.yaml
auto-ml-score --config configs/my_config.yaml
```

끝입니다.

7. 설정 파일(YAML) 이해하기

`configs/example.yaml` 의 주요 섹션을 짚어봅니다. 처음에는 기본값 그대로 두고, 데이터 경로만 바꿔서 돌려보는 것을 추천합니다.

7-1. 데이터 섹션

```
train_data_path: ./data/train.parquet      # 학습 데이터 (필수)
test_data_path:  ./data/test.parquet       # 테스트 데이터 (필수, 별도 파일)
target_column:   target                   # 0/1 타겟 컬럼 이름
features_csv:    ./configs/features.csv    # 사용할 컬럼 정의 CSV
id_columns:      [user_id]                # 스코어링 결과에 그대로 따라갈 식별자
```

💡 **주의:** 이 라이브러리는 학습 데이터를 **자동으로 train/test 분할하지 않습니다**. 사용자가 직접 별도 Parquet 파일로 준비해야 합니다.

7-2. 전처리 섹션

```
preprocessing:
  numeric_null_strategy: median          # 결측치를 무엇으로 채울지
  outlier_method: percentile             # 이상치 자르기 방법
  outlier_lower_quantile: 0.01           # 하위 1% 미만은 잘라낸다
  outlier_upper_quantile: 0.99           # 상위 1% 초과는 잘라낸다
  skew_method: signed_loglp              # 한쪽으로 치우친 분포를 펴는 방법
  skew_threshold: 1.0                   # |skew| > 1 인 컬럼만 변환
  scaling_method: standard               # 평균 0, 분산 1 로 정규화
```

처리 순서는 **결측** → **이상치** → **skew 변환** → **스케일링** 으로 고정되어 있습니다. 학습 시 사용한 통계량(중앙값, 분위수 등)은 모두 `best.joblib` 에 저장되어 스코어링 시 **동일하게 적용**됩니다.

7-3. 모델 / 튜닝 섹션

```
tuning:
  enabled: true
  n_trials: 30          # 모델당 30번 시도해서 가장 좋은 파라미터 찾기
  cv_folds: 3

models:
  lgbm:    { enabled: true, ... }
  xgb:     { enabled: true, ... }
  catboost: { enabled: true, ... }
```

세 모델을 동시에 학습한 뒤 **테스트 성능이 가장 좋은 모델 1개**가 best 로 선정됩니다. 빠르게 돌려보려면 `n_trials: 5, cv_folds: 2` 정도로 줄이세요.

7-4. 변수 선택 (Stability Selection)

무엇을 하나요?

수십~수백 개 변수 중 **진짜로 의미 있는 변수만** 자동으로 추려주는 단계입니다. 한 번 학습해 본 결과만 가지고 변수를 고르면, 운 좋게 한 번 잘 잡혔을 뿐인 변수(noise) 까지 같이 채택될 수 있습니다. 이를 막기 위해 학습 데이터의 **부분표본을 여러 번 뽑아** 부분표본마다 변수를 골라 보고, **자주 뽑히는 변수만** 최종 채택합니다. Meinshausen & Bühlmann (2010) 의 Stability Selection 절차입니다.

작동 흐름

1. 학습 데이터에서 절반씩(또는 `subsample_ratio` 비율) 부분표본을 `n_subsamples` 번 뽑습니다. 클래스 비율은 부분표본마다 유지됩니다 (stratified).
2. 부분표본마다 `base_estimator` 로 변수를 뽑습니다.
3. 각 변수가 부분표본 몇 번에서 뽑혔는지를 세서 **선택 빈도 (0.0~1.0)** 를 계산합니다.
4. `threshold` 이상으로 자주 뽑힌 변수만 채택합니다. 너무 적게 채택되면 `min_selected` 안전망이 발동해 빈도 상위 N 개를 보충합니다.

전체 설정 (모든 키)

```
feature_selection:
  enabled: true                # 사용 여부 (기본 false)
  base_estimator: lasso        # lasso | lgbm
  n_subsamples: 200            # 부분표본을 몇 번 뽑을지
  subsample_ratio: 0.5         # 부분표본 크기 비율 (원 논문 권고 0.5)
  threshold: 0.6               # 채택 임계값 (보통 0.6~0.8)
  random_state: 42             # 재현성을 위한 시드
  min_selected: 1              # 채택 변수가 부족할 때 fallback 최소 개수

# base_estimator: lasso 일 때만 사용
lasso_C: 0.1                   # L1 규제 강도. 낮을수록 더 sparse

# base_estimator: lgbm 일 때만 사용
lgbm_top_k: 30                 # 부분표본당 gain 상위 K 개 채택
lgbm_n_estimators: 100         # 부분표본 학습용 LGBM 트리 수 (가볍게)
lgbm_learning_rate: 0.1
```

base_estimator — lasso vs lgbm

항목	lasso (L1 로지스틱)	lgbm (LightGBM gain)
선택 기준	non-zero coefficient	gain 중요도 상위 K 개
잘 잡는 신호	선형 효과	비선형 · 상호작용
범주형 처리	full-X frequency encoding (1회 사전)	pandas category native
부분표본당 속도	빠름	상대적으로 느림
추천 시나리오	선형 모델 컨텍스트, 빠른 실험	트리 모델 컨텍스트, 비선형 의심

선택 기준이 다르므로 두 방식이 항상 같은 변수를 뽑지는 않습니다. 둘 다 돌려보고 채택 결과를 비교하면 도움이 됩니다.

하이퍼파라미터 가이드 (실무 기준)

- **n_subsamples** — 표준 100~500. 작은 데이터(< 5천 행)면 50~100 도 충분, 큰 데이터·고차원이면 200+. 늘릴수록 빈도 추정이 안정되지만 시간이 비례해 늘어납니다.
- **subsample_ratio** — 원 논문 권고 0.5. 너무 크면 모든 부분표본이 비슷해져 선택이 안정화되지 않고, 너무 작으면 fit 자체가 흔들립니다.
- **threshold** — 0.6 보수적, 0.8 매우 보수적. **거짓 양성**(noise 가 잘못 채택되는 일) 을 더 엄격히 막고 싶으면 0.7~0.8 로 올립니다.
- **min_selected** — 채택 변수가 부족할 때 발동하는 안전망. 모델 입력이 비어 학습이 실패하는 사고를 막습니다. 평소에는 거의 발동되지 않아야 하고 (발동 시 WARN 로그가 남습니다), 자주 발동하면 threshold 를 낮추거나 base_estimator 설정을 점검해야 합니다.
- **lasso_C** (lasso 일 때) — 규제 강도의 **역수**. 즉 **낮을수록 규제 강함** → 더 sparse 하게 잘립니다. 0.1 이 시작점, 변수가 너무 많이 살아남으면 0.01 로 조이고, 너무 잘려나가면 1.0 으로 푼다.
- **lgbm_top_k** (lgbm 일 때) — 부분표본당 채택 상한. 전체 feature 의 30~50% 정도가 적당합니다. 너무 크면 거의 모든 변수가 한 번씩 뺏혀 threshold 이상이 폭증합니다.

결과는 어디에 남나요?

- 채택된 변수 목록 (**selected_features**), 모든 변수의 선택 빈도 (**frequencies**), 실제 사용된 부분표본 수 (**n_subsamples**), fallback 발동 여부 (**fallback_used**) 가 **SelectionResult** 에 담겨 **best.joblib** 메타데이터에 저장됩니다.
- 스코어링 시 동일한 변수 셋만 모델로 흘러갑니다 — 즉 학습/스코어링 컬럼 일관성이 자동 보장됩니다.
- HTML/PDF 리포트의 **변수 선택 섹션**에 변수별 빈도 막대와 채택/제외 라벨이 표기됩니다 (예: **examples/credit/** 의 리포트 6번 섹션).

처음에는 어떻게 설정하면 좋나요?

- **변수가 10개 이하**면 굳이 안 켜도 됩니다 (**enabled: false**). 부스팅 트리 자체의 내장 selection 으로 충분합니다.
- **변수가 30개 이상**이거나 **noise 가 섞여 있을 가능성이 있으면** 다음 기본값을 추천합니다: **yaml**
feature_selection: enabled: true base_estimator: lasso n_subsamples: 100
subsample_ratio: 0.5 threshold: 0.6 lasso_C: 0.1 min_selected: 3
- 결과를 보고 채택이 너무 적으면 **threshold: 0.5** 또는 **lasso_C: 1.0** , 채택이 너무 많으면 **threshold: 0.8** 또는 **lasso_C: 0.01** 로 조정합니다.

8. features.csv 작성법

어떤 컬럼을 어떻게 쓸지 알려주는 **명단** 파일입니다. 형식은 단순합니다.

```
name,type,used
age,continuous,true
gender,category,true
```

```
income,continuous,true
internal_score,continuous,false
```

컬럼	의미
name	Parquet 파일의 컬럼명과 정확히 같아야 합니다
type	continuous (숫자) 또는 category (범주)
used	true 인 행만 학습/스코어링에 사용. false 면 제외

자주 하는 실수

- ❌ 타겟 컬럼(target)을 features.csv 에 적음 → **적지 마세요.** 타겟은 target_column 으로 따로 지정합니다.
- ❌ id 컬럼(user_id)을 features.csv 에 적음 → **적지 마세요.** id 는 id_columns 로 따로 지정합니다.
- ❌ Parquet 에 없는 컬럼명을 적음 → 학습 시 에러로 알려줍니다.

운영 팁

운영 중 어떤 변수를 잠시 빼고 싶으면, 코드를 고치지 말고 features.csv 의 해당 행만 used 를 false 로 바꾸세요.

9. 산출물(결과물) 확인하기

9-1. best.joblib (학습 산출물)

전처리기 + 모델 + 메타데이터가 하나의 파일로 묶여 있습니다. 스코어링 시 이 파일 하나만 있으면 됩니다.

9-2. HTML / PDF 리포트

artifacts/reports/report.html 을 브라우저로 열면 다음을 볼 수 있습니다:

- **모델 비교표** — 3개 모델의 성능 한눈 비교
- **튜닝 결과** — Optuna 가 찾은 최적 하이퍼파라미터
- **ROC 곡선 / PR 곡선** — 모델 분류 성능 시각화
- **Feature importance** — 어떤 변수가 중요했는지
- **Score 분포** — 예측 점수의 히스토그램
- **Confusion matrix** — 맞춘 것 / 틀린 것의 표

PDF 도 같은 내용입니다. 보고용으로 그대로 첨부할 수 있습니다.

9-3. scores.parquet (스코어링 산출물)

```
user_id, score, prediction
1001,    0.87,    1
1002,    0.12,    0
...
```

- score — 0.0 ~ 1.0 사이 확률값
- prediction — score >= threshold (기본 0.5) 면 1, 아니면 0

9-4. 로그 파일

artifacts/logs/train_YYYYMMDD_HHMMSS.log 에 실행 시작/종료 시각, 단계별 진행, 모델별 점수, 총 소요시간이 기록됩니다. 문제 발생 시 가장 먼저 보는 곳입니다.

10. 타이타닉 데이터로 검증해 보기

진짜 공개 데이터셋(891명의 생존/사망 예측)으로 전체 파이프라인을 돌려볼 수 있습니다. 약 10초 만에 ROC-AUC ≈ 0.86 의 결과를 재현합니다.

```
# 데이터 다운로드 (인터넷 필요)
python examples/titanic/prepare_data.py

# 학습 + 스코어링
auto-ml-train --config examples/titanic/config.yaml
auto-ml-score --config examples/titanic/config.yaml
```

산출물은 artifacts/titanic/ 아래에 만들어집니다.

폐쇄망에서는 환경변수 TITANIC_CSV 에 사전 배포한 CSV 경로를 지정하면 네트워크 없이 동일하게 동작합니다.

11. 자주 묻는 질문 / 문제 해결

Q1. auto-ml-train: command not found

→ 가상환경이 활성화되어 있는지 확인하세요. 프롬프트 앞에 (.venv) 가 보여야 합니다. 안 보이면 다시 source .venv/bin/activate .

Q2. ValueError: target column must contain only 0 and 1

→ 타겟 컬럼에 0/1 외의 값(NaN, 2, "yes" 등)이 섞여 있습니다. 데이터를 정리한 뒤 다시 시도하세요.

Q3. ValueError: column X is all NaN

→ 어떤 수치형 컬럼이 학습 데이터에서 **전부 결측**이라 처리할 수가 없는 상태입니다. 해결 방법은 셋 중 하나:

- features.csv 에서 해당 컬럼의 `used` 를 `false` 로
- 결측 처리 전략을 `constant` 로 바꿔 고정값 대입
- 데이터 자체를 정리해서 다시 만들기

Q4. PDF 리포트가 안 만들어져요

→ WeasyPrint 시스템 의존성이 빠져서 그렇습니다. HTML 은 정상 생성됩니다. PDF 가 꼭 필요하다면 [3. 사전 준비](#) 를 참조해 시스템 패키지를 설치하세요. 또는 설정에서 `reporting.generate_pdf: false` 로 끄면 됩니다.

Q5. 학습이 너무 오래 걸려요

→ 다음을 줄여 보세요:

```
tuning:
  n_trials: 5          # 30 → 5
  cv_folds: 2          # 3 → 2
training:
  cv_folds: 3          # 5 → 3
```

Q6. 모델 한 개만 쓰고 싶어요

→ `models.<이름>.enabled: false` 로 끄면 됩니다. 단, 최소 1개는 켜져 있어야 합니다.

Q7. 임계값을 0.5 가 아닌 다른 값으로 쓰고 싶어요

→ `scoring.threshold` 를 바꿉니다. 예: `0.3` 으로 두면 0.3 이상이면 1로 분류.

Q8. 새로 들어온 데이터에 컬럼 하나가 없어요

→ 학습 시 사용한 features 정의가 `best.joblib` 메타데이터에 박혀 있어, 스코어링 입력에 같은 컬럼 셋이 있어야 합니다. 컬럼이 빠졌으면 스코어링 단계에서 검증 에러로 알려줍니다.

12. 폐쇄망(인터넷이 안 되는 환경) 이관

다음 4개만 옮기면 됩니다:

1. 패키지 wheel 묶음 — 외부망에서 미리 만들어 둡니다. `bash pip wheel -r requirements.txt -w wheelhouse/`
2. 본 저장소 자체 — `git clone` 한 폴더 그대로.
3. 학습 산출물 — `artifacts/models/best.joblib`
4. 스코어링용 설정 YAML

폐쇄망에서는 다음과 같이 설치합니다:

```
pip install --no-index --find-links=wheelhouse -r requirements.txt
pip install -e .
auto-ml-score --config configs/my_config.yaml
```

13. 용어 사전

용어	설명
이진 분류	0/1 (참/거짓, 양/음) 두 가지 중 하나를 맞추는 문제
타겟 (target)	예측하고 싶은 답 컬럼 (0 또는 1)
피처 (feature)	예측의 근거가 되는 입력 변수들
전처리 (preprocessing)	결측 채우기, 이상치 자르기, 스케일링 등 학습 전 데이터 정리
스케일링	변수마다 다른 단위(나이 vs 소득)를 비슷한 크기로 맞추는 작업
이상치 (outlier)	분포에서 너무 동떨어진 극단값
Skew (왜도)	분포가 한쪽으로 치우친 정도. 큰 값을 log 등으로 퍼주면 학습이 안정됨
Stability Selection	부분표본을 여러 번 뽑아 자주 선택되는 변수만 추리는 방법 (Meinshausen & Bühlmann, 2010). <code>feature_selection</code> 섹션 참고
Optuna / TPE	베이지안 방식으로 하이퍼파라미터를 똑똑하게 탐색하는 라이브러리
하이퍼파라미터	모델 학습 전에 사람이 정해주는 설정값 (학습률, 트리 깊이 등)
KFold / OOF	데이터를 K 등분해 돌아가며 평가. Out-Of-Fold 예측을 모아 점수 계산
ROC-AUC	분류 성능 지표 (0.5=찍기, 1.0=완벽). 이 라이브러리의 기본 지표
Artifact	학습 결과물(모델 + 전처리기 + 메타데이터)을 하나로 묶은 파일

용어	설명
Parquet	컬럼 기반 효율적 데이터 파일 형식 (CSV 보다 빠르고 작음)

마치며

이 가이드는 **시작용**입니다. 더 깊이 들어가고 싶다면:

- 전체 설정 옵션: `configs/example.yaml`
- 코드에서 직접 호출: `examples/run_train.py`, `examples/run_score.py`
- 내부 dataclass 정의: `auto_ml/config.py`
- 상세 동작 원리: `README.md`

문제가 풀리지 않으면 `artifacts/logs/` 의 최신 로그 파일과 함께 담당자에게 공유해 주세요. 즐거운 모델링 되세요!